Virtual Memory & IPC

A brief intro of virtual memory
Inter process communication

- Mmap
- Pipes

You will see more details of virtual memory in BIL 301 OS

- In simple embedded systems and early computers,
  - processes directly access memory –
    - “Address 1234” cor responds to a particular byte stored in a particular part of physical memory.
  - For example, the IBM 709 had to read and write directly to tape with no level of abstraction
- In modern computers,
  - there is a translation process between the address of a particular CPU instruction or piece of data of a process and the actual byte of physical memory (“RAM”).
  - Memory addresses no longer map to physical addresses
  - The process runs inside virtual memory.

The modern process of translating memory is as follows.

1. A process makes a memory request
2. The circuit first checks the Translation Lookaside Buffer (TLB) if the address page is cached into memory. It skips to the reading from/writing to phase if found otherwise the request goes to the MMU.
3. The Memory Management Unit (MMU) performs the address translation. If the translation succeeds, the page gets pulled from RAM – conceptually the entire page isn’t loaded up. The result is cached in the TLB.
4. The CPU performs the operation by either reading from the physical address or writing to the address.

Address translation

The Memory Management Unit is part of the CPU, and it converts a virtual memory address into a physical address.

- consider a 32-bit machine, meaning pointers are 32-bits.
  - They can address 2^32 different locations or 4GB of memory where one address is one byte.

Imagine we had a large table for every possible address where we will store the real i.e. physical address.

- Each physical address will need 4 bytes–to hold the 32-bits.
- Naturally, this scheme would require 16 billion bytes to store all of the entries.
- It should be painfully obvious that our lookup scheme would consume all of the memory that we could buy for our 4GB machine.

The solution is to chunk memory into small regions called **‘pages’** and **‘frames’** and use a lookup table for each page.

Terminology

A **page** is a block of virtual memory.

- A typical block size on Linux is 4KiB
  - or 2^12 addresses,
- one can find examples of larger blocks

a **frame** or sometimes called a **‘page frame’** is a block of physical memory or RAM – Random Access Memory.

- A frame is the same number of bytes as a virtual page or 4KiB on our machine.

Page Number	Frame Number
0	42
1	30
2	24
•••	•••

Figure 9.1: Explicit Frame Table

Page table

- 32-bit computers
  - 2^32 address/2^12(address/page) = 2^20 pages.

- 64-bit computer
  - 2^64 address/2^12(address/page) = 2^52 pages =~ 10^15

- To access a particular byte in a frame, an MMU goes from the start of the frame and adds the offset

Page Number	Frame Number
0	42
1	30
2	24
•••	•••

We will assume that a 32-bit machine has 4KiB pages

2^20 pages, so first 20 bits is the page number,

The rest is the offset (address-index in the page)

VirtualAddress = 11110000111100001111000010101010 (binary)

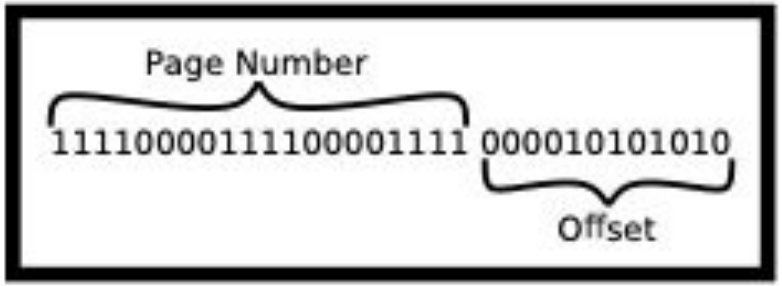


Figure 9.2: Splitting Address

One level dereferencing

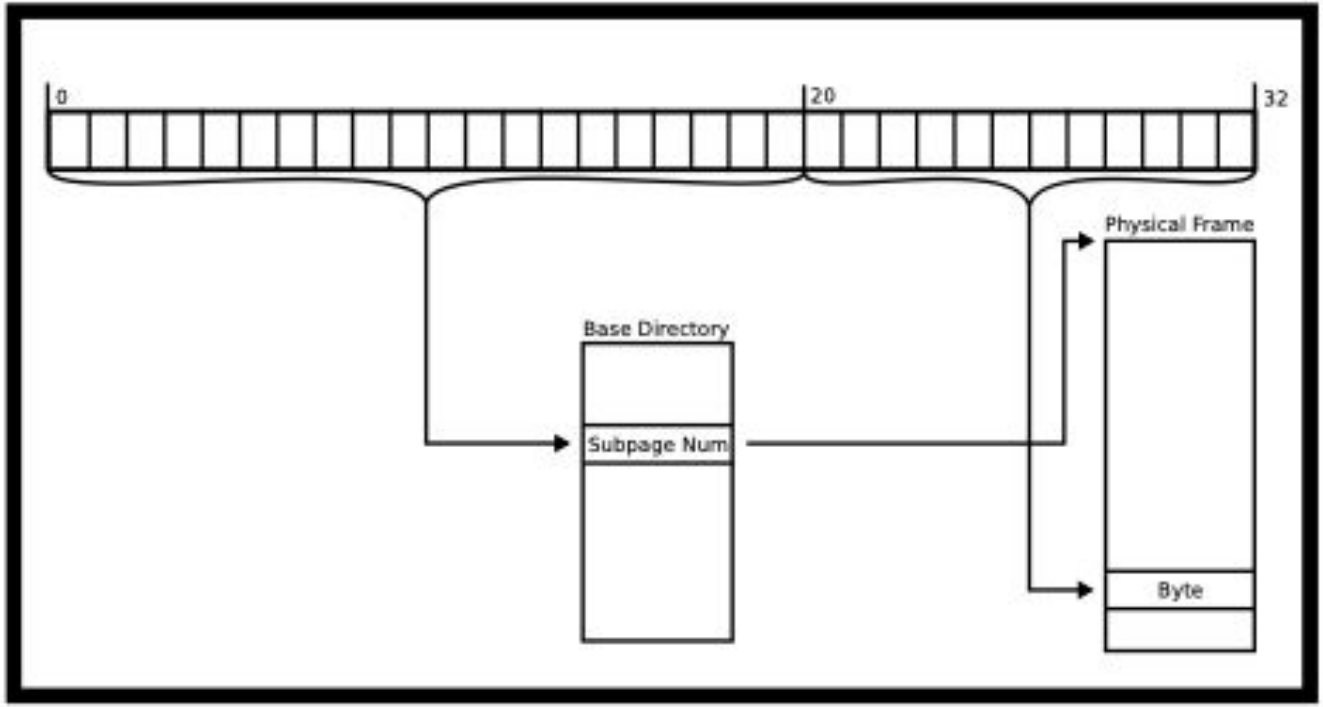


Figure 9.3: One level dereference

Example: What is the page table size?

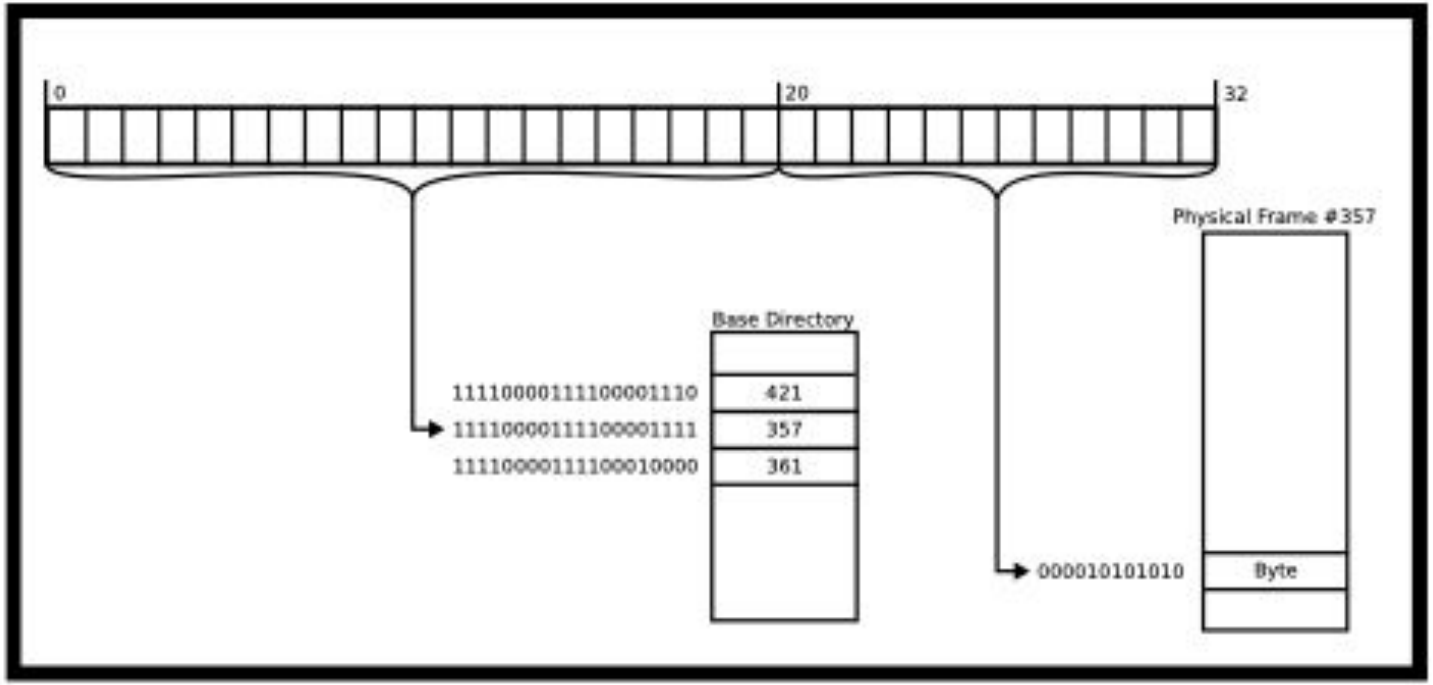


Figure 9.4: One level dereference example

Multi level page tables to reduce page-table size

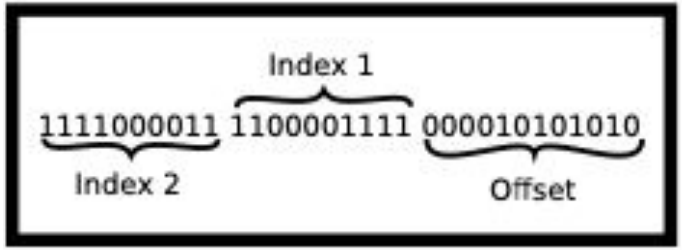


Figure 9.5: Three Way Address Split

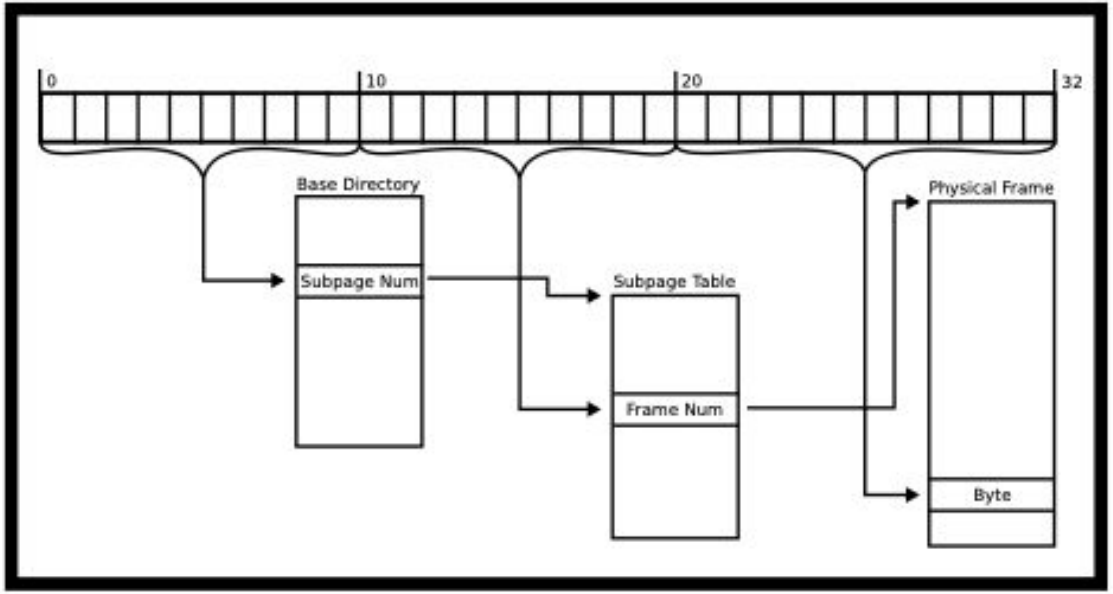


Figure 9.6: Full page table dereference

Multi-level page tables only allocates page-table space in proportion to the amount of address space you are using;

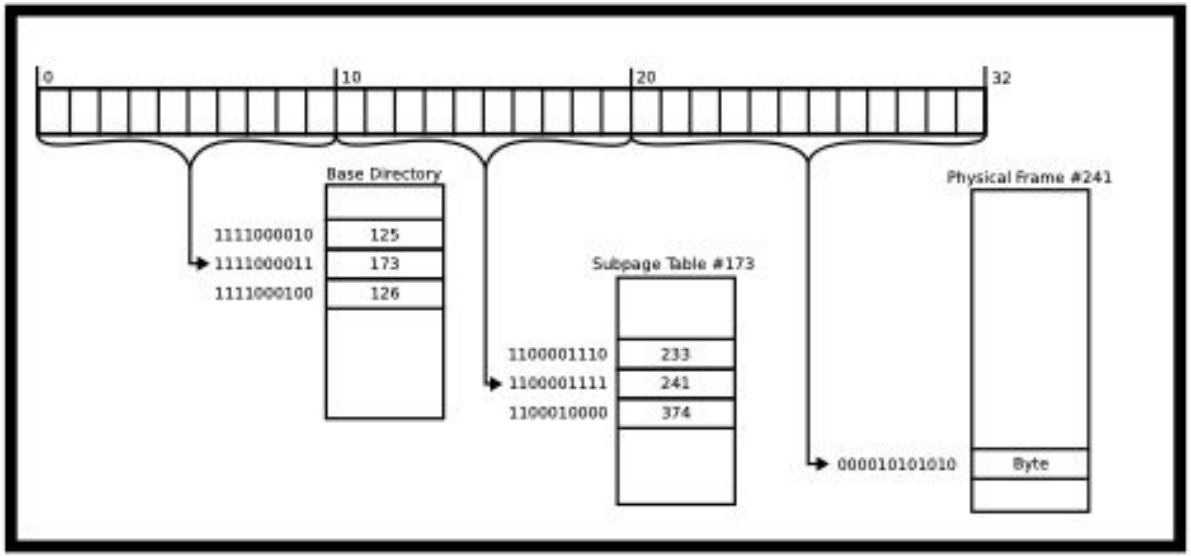


Figure 9.7: Full page example dereference

Page Table Disadvantages & TLB (translation lookaside table)

- Page tables are slow.
  - For a single page table, our machine is now twice as slow! Two memory accesses are required.
  - For a two-level page table, memory access is now three times as slow – three memory accesses are required.
- To overcome this overhead, the MMU includes an associative cache of recently-used virtual-page-to-frame lookups. This cache is called the TLB (“translation lookaside buffer”).
- Every time a virtual address needs to be translated into a physical memory location, the TLB is queried in parallel to the page table.
- For most memory accesses of most programs, there is a significant chance that the TLB has cached the results. However, if a program has inadequate cache coherence, the address will be missing in the TLB, meaning the MMU must use the much slower page table translation.

MMU algorithm

here is a sort of pseudocode associated with the MMU. We will assume that this is for a single-level page table.

1. Receive address

2. Try to translate address according to the programmed scheme

3. If the translation fails, report an invalid address

4. Otherwise,

(a) If the TLB contains the physical memory, get the physical frame from the TLB and perform the read and write.

(b) If the page exists in memory, check if the process has permissions to perform the operation on the page meaning the process has access to the page, and it is reading from the page/writing to a page that it has permission to do so.

i. If so then do the dereference provide the address, cache the results in the TLB
ii. Otherwise, trigger a hardware interrupt. The kernel will most likely send a SIGSEGV or a Segmentation Violation.

(c) If the page doesn't exist in memory, generate an Interrupt.

i. The kernel could realize that this page could either be not allocated or on disk. If it fits the mapping, allocate the page and try the operation again.
ii. Otherwise, this is invalid access and the kernel will most likely send a SIGSEGV to the process.

How would you alter this for a multi-level page table?

Bits on pages

This is heavily dependent on the chipset. We will include some bits that have historically been popular in chipsets.

1. **The read-only bit** marks the page as read-only. Attempts to write to the page will cause a page fault.

2. **The execution bit** defines whether bytes in a page can be executed as CPU instructions.

- Processors may merge these bits into one and deem a page either writable or executable.
- This bit is useful because it prevents stack overflow or code injection attacks when writing user data into the heap or the stack because those are not read-only and thus not executable.

3. **The dirty bit** allows for performance optimization.

And many others

Page faults

A page fault may happen when a process accesses an address in a frame missing in memory. There are three types of Page Faults

1. Minor If there is no mapping yet for the page, but it is a valid address.

- This could be memory asked for by sbrk(2) but not written to yet meaning that the operating system can wait for the first write before allocating space – if it was read from, the operating system could short circuit the operation to read 0. The OS simply makes the page, loads it into memory, and moves on.

2. Major If the mapping to the page is exclusively on disk.

- The operating system will swap the page into memory and swap another page out. If this happens frequently enough, your program is said to thrash the MMU.

3. Invalid When a program tries to write to a non-writable memory address or read to a non-readable memory address.

- The MMU generates an invalid fault and the OS will usually generate a SIGSEGV meaning segmentation violation meaning that the program wrote outside the segment that it could write to.

What does this have to do with IPC?

Before, we knew that processes had isolation.

1. you didn’t know how that isolation mapped.
2. you may not know how you can break this isolation.

Interprocess communication requires “breaking” this isolation in some way.

What does this have to do with IPC?

To break a memory level isolation,

we have two avenues.

- One is to ask the kernel to provide some kind of interface.
- The other is to ask the kernel to map two pages of memory to the same virtual memory area and handle all the synchronization yourself.

Mmap: map or unmap files or devices into memory

- mmap is a trick of virtual memory
  - instead of mapping a page to a frame, it can be mapped to a file on disk,
  - and the frame can be shared among processes.
- We can use **mmap** to read from a file on disk efficiently or sync changes to the file.
  - One of the big optimizations is a file may be **“lazily”** allocated to memory.

```
char buf[100] = "merhaba";
int len  = strlen(buf);
int offset = 0;
int fd = open("hello.txt", O_WRONLY);
lseek(fd, offset, SEEK_SET);
write(fd, buf, len);
close(fd);
```

```c
/*standard file I/O*/
char buf[100] = "merhaba";
int len  = strlen(buf);
int offset = 0;
int fd = open("hello.txt", O_WRONLY);
lseek(fd, offset, SEEK_SET);
write(fd, buf, len);
close(fd);
```

```c
/*with mmap*/
#include <sys/mman.h>
#include <fcntl.h> #include <unistd.h>
#include <stdio.h>
int main(){

   int len  = 100;
   off_t offset = 0;
   int fd2 = open("hello.txt", O_RDWR);
   char *addr = mmap(NULL, len, PROT_WRITE,
                     MAP_SHARED, fd2, offset);
   addr[0] = 'A';
   addr[1] = 'B';
   printf("%.100s\n", addr);
   munmap(addr, len);
   close(fd2);
}
```

mmap

```
int len  = 100;
off_t offset = 0;

int fd2 = open("hello.txt", O_RDWR);
char *addr = mmap(NULL, len, PROT_WRITE,
                  MAP_SHARED, fd2, offset);

addr[0] = 'A';
addr[1] = 'B';
printf("%.100s\n", addr);
munmap(addr, len);
close(fd2);
```

The kernel sees that the program wants to mmap the file into memory, so it will reserve some space in your address space that is the length of the file.

That means when the program writes to **addr[0]** that it writes to the first byte of the file.

The kernel can do some optimizations too. Instead of loading the whole file into memory, it may only load pages at a time.

A program may only access 3 or 4 pages making loading the entire file a waste of time. Page faults are so powerful because let the operating system take control of when a file is used.

mmap

```
#include <sys/mman.h>
void *mmap(void *addr, size_t length,
           int prot, int flags,
           int fd, off_t offset);
```

- `addr`,
  - Start address of new mapping
  - If NULL, kernel chooses
- `length`,
  - `Length` of the mapping in bytes starting from `offset`
-
- `prot` determines the type of access
  - **PROT_READ** This means the process can read the memory. The underlying file descriptor, in this case, must be opened with read privileges.
  - **PROT_WRITE** This means the process can write to the memory.
  - **PROT_EXEC** This means the process can execute this piece of memory.
  - **PROT_NONE** This means the process can't do anything with the mapping. This could be useful if you implement guard pages in terms of security.
- `flags` determines how to handle the changed data
  - **MAP_SHARED** This mapping will be synchronized to the underlying file object. The file descriptor must've been opened with write permissions in this case.
  - **MAP_PRIVATE** This mapping will only be visible to the process itself. Useful to not thrash the operating system.

```c
#include <sys/mman.h>
void *mmap(void addr[.length],
       size_t length, int prot, int flags,
       int fd, off_t offset);
```

Fd is a file descriptor can be

- `fd = open("hello.txt", O_WRONLY);`
- `fd = shm_open(shmpath, O_CREAT | O_EXCL | O_RDWR, 0600);`

https://man7.org/linux/man-pages/man7/shm_overview.7.html

General guidance for memory-mapped files

1. Get a file descriptor with a call to the **open** or **shm_open** function.
2. Map the object using the file descriptor with a call to the **mmap** function.
3. Unmap the object with a call to the **munmap** function.
4. Close the object with a call to the **close** function.
5. Remove the shared-memory object with a call to the shm_unlink function or optionally remove a memory-mapped file with a call to the unlink function.

Taken from https://www3.physnet.uni-hamburg.de/physnet/Tru64-Unix/HTML/APS33DTE/DOCU_004.HTM

Walkthrough of the example code in the man pages

Our command-line utility will take a file, offset, and length to print.

To obtain file info in linux

```
#include <sys/stat.h>
struct stat {
   dev_t     st_dev;         /* ID of device containing file */
   ino_t     st_ino;         /* Inode number */
   mode_t    st_mode;        /* File type and mode */
   nlink_t   st_nlink;       /* Number of hard links */
   uid_t     st_uid;         /* User ID of owner */
   gid_t     st_gid;         /* Group ID of owner */
   dev_t     st_rdev;        /* Device ID (if special file) */
   off_t     st_size;        /* Total size, in bytes */
   blksize_t st_blksize;     /* Block size for filesystem I/O */
   blkcnt_t  st_blocks;      /* Number of 512B blocks allocated
*/
...
```

```
int fd = open("hello.txt", O_RDWR);
struct stat sb;
fstat(fd, &sb);
/*
sb.st_size is the file size
*/
```

mmap doesn’t let the program pass in any value as an offset, it needs to be a multiple of the page size.

```
  off_t offset = atoi(argv[2]);
  pa_offset = offset & ~(sysconf(_SC_PAGE_SIZE) - 1);
   /* offset for mmap() must be page aligned, it rounds down the value:
example
Let Page size be 4096

~(4096-1) = ~4095 = -4096 that is  1111...1000000000000 which zeros the lowest bits of

offset*/
```

```
char * addr = mmap(NULL,
    length + offset - page_offset,
    PROT_READ, MAP_PRIVATE,
    fd, page_offset);
```

1. **NULL**, this tells mmap we don't need any particular address to start from

2. **length + offset - page_offset**, mmaps the "rest" of the file into memory (starting from offset)

3. **PROT_READ**, we want to read the file

4. **MAP_PRIVATE**, tell the OS, we don't want to share our mapping

5. **fd**, object descriptor that we refer to

6. **pa_offset**, the page aligned offset to start from

```
write(1, addr + offset - page_offset, length);
munmap(addr, length + offset - pa_offset);
close(fd);
```

we can interact with the address as if it were a normal buffer.

After, we have to unmap the file and close the file descriptor to make sure other system resources are freed.

Using mmap to communicate across processes

```
/*allocate some space with mmap*
int size = 100 * sizeof(int);
void *addr = mmap(0, size, PROT_READ | PROT_WRITE,
                  MAP_SHARED | MAP_ANONYMOUS, -1, 0);
int *shared = addr;
```

- MAP_ANONYMOUS :
  - **The mapping is not backed by any file;**
  - its contents are initialized to zero.
  - The fd argument is ignored; however, some implementations require fd to be -1
  - The offset argument should be zero.

```c
pid_t child = fork();
if (child > 0){
    shared[0] = 10;
    shared[1] = 20;
}
else{
    sleep(1); // Check the synchronization chapter for a better way
    printf("%d\n", shared[1] + shared[0]);
}
munmap(addr, size);
return 0;
```

Since sleep is not a mutex there is no assurance that the values are communicated (use semaphores)

Note that each process should call munmap when done using the piece of memory.

MMAP for Two different process (not parent-child)

Use **shm_open** function.

```
int
    shm_open(const char *name, int oflag, ...);
```

The shared memory object referenced by **name** is opened for reading and/or writing as specified by the argument **oflag** and the file descriptor returned to the calling process.

Repeated calls to **shm_open**
with the same string value for **name**() will return a file descriptor referring to the same shared memory object, provided that the object has not been unlinked by a call to **shm_unlink**().

See man shm_open

Example (from man shm_open)

1 process writes data to shared memory, the other one converts it to uppercase

- 1 need to wait for the data
- 1 need to wait for the uppercase

```
struct shmbuf {
   sem_t  sem1;            /* POSIX unnamed semaphore */
   sem_t  sem2;            /* POSIX unnamed semaphore */
   size_t cnt;             /* Number of bytes used in 'buf' */
   char   buf[BUF_SIZE];   /* Data being transferred */
};
```

```c
/* pshm_ucase_bounce.c          Licensed under GNU General Public License v2 or later.      */
#include <ctype.h>
#include "pshm_ucase.h"
int main(int argc, char *argv[]) {
   if (argc != 2) {       fprintf(stderr, "Usage: %s /shm-path\n", argv[0]);       exit(EXIT_FAILURE);   }
   char *shmpath = argv[1];

    /* Create shared memory object and set its size to the size     of our structure. */
    int fd = shm_open(shmpath, O_CREAT | O_EXCL | O_RDWR, S_IRUSR | S_IWUSR);
    if (fd == -1)    errExit("shm_open");

    if (ftruncate(fd, sizeof(struct shmbuf)) == -1)      errExit("ftruncate");

    /* Map the object into the caller's address space. */
    struct shmbuf *shmp = mmap(NULL, sizeof(*shmp), PROT_READ | PROT_WRITE,
                               MAP_SHARED, fd, 0);

    if (shmp == MAP_FAILED)    errExit("mmap");

    /* Initialize semaphores as process-shared, with value 0. */
    if (sem_init(&shmp->sem1, 1, 0) == -1)       errExit("sem_init-sem1");
    if (sem_init(&shmp->sem2, 1, 0) == -1)       errExit("sem_init-sem2");
```

```
/* Wait for 'sem1' to be posted by peer before touching  shared memory. */
   if (sem_wait(&shmp->sem1) == -1)   errExit("sem_wait");

   /* Convert data in shared memory into upper case. */
   for (int j = 0; j < shmp->cnt; j++)
       shmp->buf[j] = toupper((unsigned char)shmp->buf[j]);

   /* Post 'sem2' to tell the peer that it can now  access the modified data in shared
memory. */
   if (sem_post(&shmp->sem2) == -1)      errExit("sem_post");

   /* Unlink the shared memory object. Even if the peer process
       is still using the object, this is okay. The object will
       be removed only after all open references are closed. */
   shm_unlink(shmpath);
   exit(EXIT_SUCCESS);
}
```

```c
/* pshm_ucase_send.c            Licensed under GNU General Public License v2 or later.          */
#include <string.h>
#include "pshm_ucase.h"

int main(int argc, char *argv[]) {
   if (argc != 3) {     fprintf(stderr, "Usage: %s /shm-path string\n", argv[0]); exit(EXIT_FAILURE);  }
   char *shmpath = argv[1];   char *string = argv[2];
   size_t len = strlen(string);
   if (len > BUF_SIZE) {       fprintf(stderr, "String is too long\n");       exit(EXIT_FAILURE); }

   /* Open the existing shared memory object and map it into the caller's address
space. */
   int fd = shm_open(shmpath, O_RDWR, 0);
   if (fd == -1)    errExit("shm_open");

   struct shmbuf *shmp = mmap(NULL, sizeof(*shmp), PROT_READ | PROT_WRITE,
                              MAP_SHARED, fd, 0);
   if (shmp == MAP_FAILED)
       errExit("mmap");
```

```
    /* Copy data into the shared memory object. */
    shmp->cnt = len;
    memcpy(&shmp->buf, string, len);
    /* Tell peer that it can now access shared memory. */
    if (sem_post(&shmp->sem1) == -1)
        errExit("sem_post");
    /* Wait until peer says that it has finished accessing
       the shared memory. */
    if (sem_wait(&shmp->sem2) == -1)
        errExit("sem_wait");
    /* Write modified data in shared memory to standard output. */

    write(STDOUT_FILENO, &shmp->buf, len);
    write(STDOUT_FILENO, "\n", 1);

    exit(EXIT_SUCCESS);
}
```

Sharing anonymous memory is an efficient form of inter-process communication because there is no copying, system call, or disk-access overhead - the two processes share the same physical frame of main memory.

On the other hand, shared memory, like in a multithreading context, creates room for data races. Processes that share writable memory might need to use synchronization primitives like mutexes to prevent these from happening.

Pipes

You've seen the virtual memory way of IPC, but there are more standard versions of IPC that are provided by the kernel. One of the big utilities is POSIX pipes. A pipe simply takes in a stream of bytes and spits out a sequence of bytes.

pipes were invented to take the output of one program and feed it to the input of another program though they have other uses today

```
$ ls -l | more
```

```
$ ls -1 | cut -d'.' -f1 | sort | uniq | tee dirents
```

What does the following code do?

- First, it lists the current directory.
  - The -1 means that it outputs one entry per line.
- The **cut** command then takes everything before the first period.
- **sort** sorts all the input lines,
- uniq makes sure all the lines are unique.
- Finally, **tee** outputs the contents to the file **dirents** and the terminal for your perusal.
- The important part is that bash creates 5 separate processes and connects their standard outs/stdins with pipes the trail looks something like this.

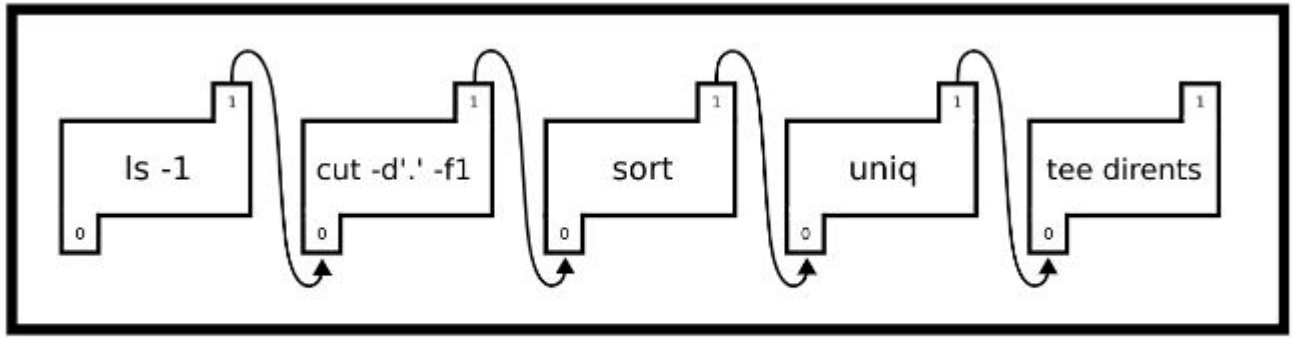


Figure 9.8: Pipe Process Filedescriptor redirection

The numbers in the pipes are the file descriptors for each process and the arrow represents the redirect or where the output of the pipe is going.

POSIX pipes: pipe() system call

A POSIX pipe is almost like its real counterpart - a program can stuff bytes down one end and they will appear at the other end in the same order.

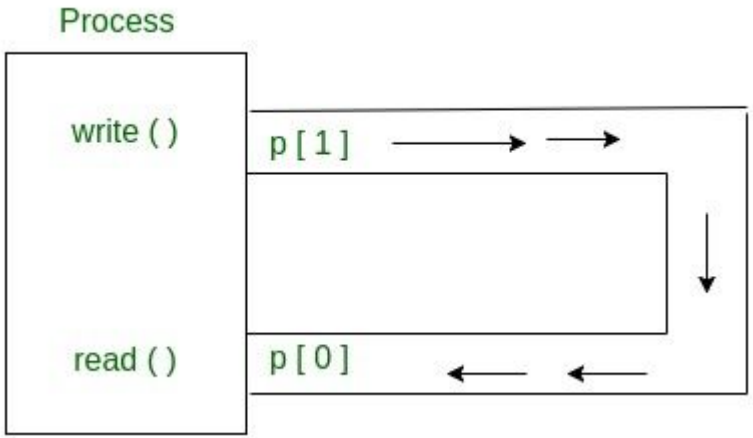
Unlike real pipes, however, the flow is always in the same direction, one file descriptor is used for reading and the other for writing.

The pipe system call is used to create a pipe. These file descriptors can be used with **read** and **write**.

A common method of using pipes is to create the pipe before forking to communicate with a child process

```
#include <unistd.h>

int pipe(int p[2]);
```

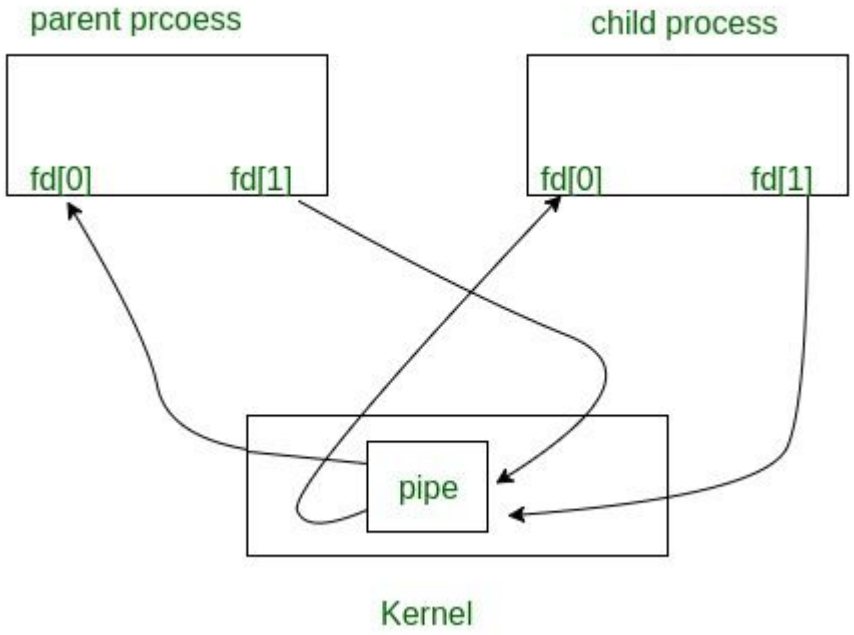


https://www.geeksforgeeks.org/pipe-system-call/

```
int fd[2];
pipe(fd);
/*fd[0] is set up for reading,
fd[1] is set up for writing*/


pid_t child = fork();
```

- file descriptors remain open across child and parent processes.
- If we call fork after creating a pipe, then the parent and child can communicate via the pipe.



https://www.geeksforgeeks.org/pipe-system-call/

```c
#include <unistd.h>
#include <stdio.h>
int main(){
   int filedes[2];
   pipe(filedes);
   pid_t child = fork();
   if (child > 0){ /* I must be the parent */
       char buffer[80];
       int bytesread = read(filedes[0], buffer, sizeof(buffer));
       // do something with the bytes read
   }else{
       write(filedes[1], "done", 4);
   }
}
```

infinite loop!

```
int main(){
    int fd[2]; pipe(fd);
    printf("Reading from %d, writing to %d\n", fd[0], fd[1]);
    if (fork() > 0){/*the parent */
        write(fd[1], "Hi Child!", 9);
        wait(NULL);
    }else{
        char buf; int bytesread;
        // read one byte at a time.
        while ((bytesread = read(fd[0], &buf, 1)) > 0){
            putchar(buf);
        }
    }
    return 0;
```

When there are no characters, child blocks and waits more character because the pipe is not closed.

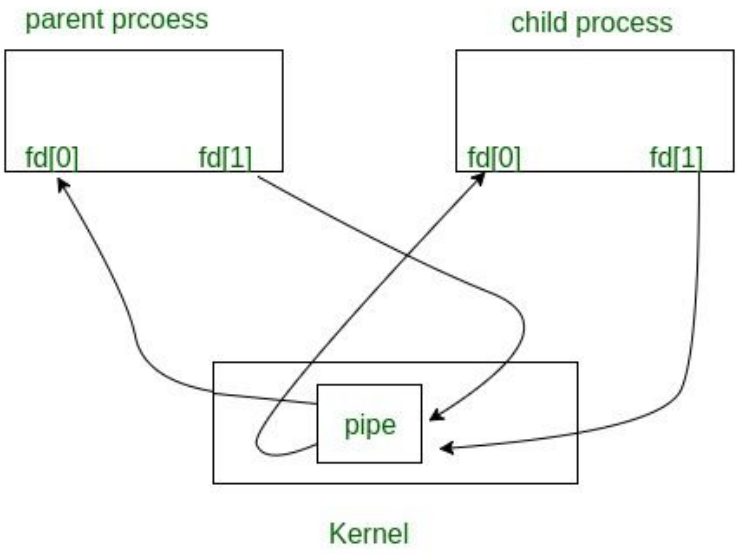
A simple solution

```
while ((bytesread = read(fd[0], &buf, 1)) > 0){
    putchar(buf);
    /* a solution */
    if (buf == '!') break; /* End of message */

}
```

Or close all writers

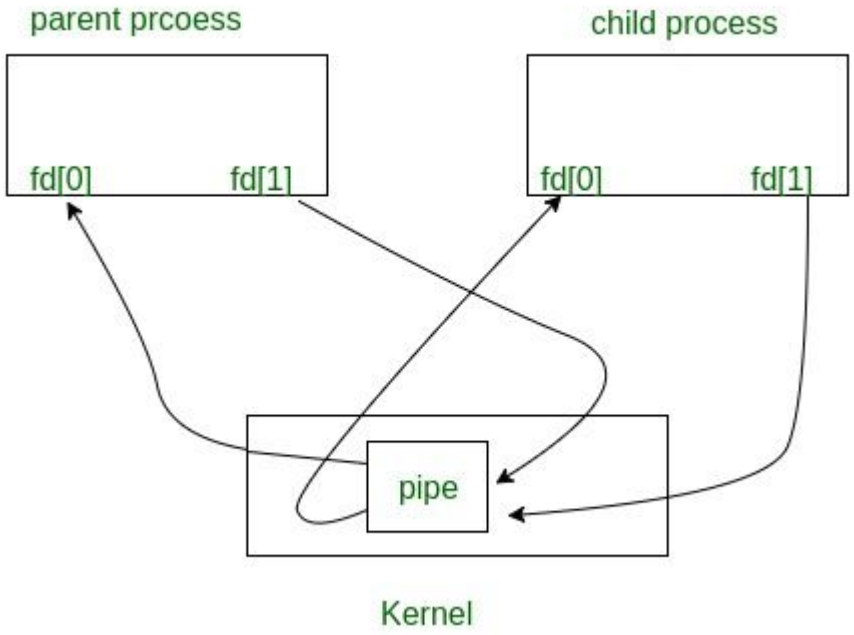
```
if (p > 0){
        write(fd[1], "Hi Child!", 9);
        close(fd[1]);
        wait(NULL);
    }
    else{
        char buf;
        int bytesread;
        close(fd[1]);
        while ((bytesread = read(fd[0], &buf, 1)) > 0){
            putchar(buf);
        }
```



If there is no readers!

If all file descriptors referring to the read end of a pipe have been closed,

then a write(2) will cause a **SIGPIPE** signal to be generated for the calling process.



```c
#include <stdio.h>
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
void no_one_listening(int signal){
   write(1, "No one is listening!\n", 21);
}
```

```c
int main(){
   signal(SIGPIPE, no_one_listening);
   int filedes[2];
   pipe(filedes);
   pid_t child = fork();
   if (child > 0){/*  the parent.*/
   /* Close the listening end of the pipe */
       close(filedes[0]);
   }else{
       close(filedes[0]);
       /* Child writes messages to the pipe */
       write(filedes[1], "One", 3);
       write(1, "Done\n", 5);
   }
   return 0;
}
```

Duplicate file descriptor

```
int dup( int oldfd );
int dup2( int oldfd, int newfd );
```

The dup() system call creates a copy of the file descriptor

```
int pipe2(int pipefd[2], int flags);
```

- **flags**
  - **O_CLOEXEC**
    - Sets the close-on-exec flag on the two new file descriptors.

$ ls -1 | cut -f1 -d.

- This command takes the output of ls -1 which lists the content of the current directory on one line each and pipes it to cut.
- Cut take a delimiter, in this case, a dot, and a field position, in our case 1, and outputs per line the nth field by each delimiter.
- At a high level, this grabs the file names without the extension of our current directory.
-

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>
int main(){
   int pipe_fds[2];
   /* Call with the O_CLOEXEC flag to
prevent any commands from blocking */
  pipe2(pipe_fds, O_CLOEXEC);
   /*Remember for pipe_fds,
   the program read then write
   (reading is 0 and writing is 1)*/
```

```c
if (!fork()){/* Child */
        /* Make the stdout of the process,
         the write end */
        dup2(pipe_fds[1], 1);
        /* Exec! Don't forget the cast*/
        execlp("ls", "ls", "-1", (char *)NULL);
        exit(-1);
    }
    /*parent: the stdin of the process is
      the read end */
    dup2(pipe_fds[0], 0);
    execlp("cut", "cut", "-f1", "-d.", (char *)NULL);
    exit(-1);
    return 0;
}
```

Named pipes

See man mkfifo

```
#include <sys/types.h>#include <sys/stat.h>

int mkfifo(const char *pathname, mode_t mode);
```